Introduction to SQL and Advanced Functions Assignment

Question 1 : Explain the fundamental differences between DDL, DML, and DQL commands in SQL. Provide one example for each type of command.

Answer: DDL,DML,and DQL are various commands in Sql.

- DDL(Data Definition Language) manages the database containers and structure. It defines and modifies the database structures and schemas. Such as change the architrectural layout ,not the values of the schema.
  Various DDL commands are :
  1. Create
  2. Drop
  3. Truncate
  4. Alter
  5. Rename

- DML(Data Manipulation Language) manages the records stored inside that container. It inserts,updates and removes thre row-level records. Act on individual rows or tuples inside a table.
  Various DML commands are :
  1. Insert
  2. Delete
  3. Update

- DQL(Data Query Language) extracts information out of it without modifying anything. Used to fetch and presents stored records to the user.

The DQL command used in sql is :

1. Select

Question 2 : What is the purpose of SQL constraints? Name and describe three common types of constraints, providing a simple scenario where each would be useful.

Answer: SQL constraints are the rules applied to table columns or rows to ensure data integrity, accuracy, and reliability. They automatically block invalid data entries (such as duplicates or out-of-range values) from being saved, meaning the database guarantees data consistency regardless of how the application writes it.

Three common types of SQL constraints are :

1. PRIMARY KEY uniquely identifies each record (row) in a table. It is a combination of a NOT NULL and UNIQUE constraint, meaning every row must have a value, and that value cannot be repeated. A table can only have one primary key.

- **Scenario:** In an Employees table, setting employee_id as the primary key ensures that two different employees are never accidentally assigned the exact same ID, preventing confusion in payroll or HR systems.

2. UNIQUE ensures that all values in a specific column are distinct from one another across all rows in the table. Unlike a primary key, it can allow NULL values.

- **Scenario:** In a Users table, applying a UNIQUE constraint to the email_address column prevents two different users from creating an account using the same email address.

3. CHECK limits the values that can be inserted into a column by applying a specific condition or logical rule. If the condition evaluates to true, the data is accepted; otherwise, the transaction fails.

- **Scenario:** In a Products table, adding a CHECK (price >= 0) constraint stops accidental insertion of negative prices, saving the system from bad data.

Question 3 : Explain the difference between LIMIT and OFFSET clauses in SQL. How would you use them together to retrieve the third page of results, assuming each page has 10 records?

Answer: In SQL, the LIMIT clause restricts the maximum number of rows returned by a query, while the OFFSET clause specifies how many rows to skip before starting to return

data.

To fetch the third page of results, calculate your offset based on the previous pages.

- **Page 1**: Displays records 1 to 10 (Skips 0 records : OFFSET 0)
- **Page 2**: Displays records 11 to 20 (Skips 10 records : OFFSET 10)
- **Page 3**: Displays records 21 to 30 (Skips 20 records : OFFSET 20)

And limit will be set to 10 as the 3rd page will have 10 records.

 # syntax:

```
Select * from table_name
Limit 10
Offset 20;
```

Question 4 : What is a Common Table Expression (CTE) in SQL, and what are its main benefits? Provide a simple SQL example demonstrating its usage.

Answer: A Common Table Expression (CTE) is a temporary, named result set in SQL. It is defined using the "with" clause. It exists only during the execution of a single query statement. a virtual table that acts as an alternative to complex nested subqueries.

Main Benefits of Using a CTE :

- Improved Readability: Breaks large, queries into clean, modular steps.
- Code Reusability: the same CTE can be referenced multiple times within the final query.
- Recursive Queries: Enables querying hierarchical data structures like organization charts.

- Easier Maintenance: Simplifies debugging because you can isolate parts of the query.
- Alternative to Views: Serves as a quick, local substitute when a permanent view isn't needed.

A simple example to show common table expressions :

```
-- Define the Common Table Expression
WITH High_Earners  AS
( SELECT Employee_ID, FirstName, LastName, Department, Salary

FROM Employees
WHERE Salary > 80000 )

-- Use the CTE in the main query

SELECT FirstName, LastName, Salary
 FROM HighEarners
WHERE Department = 'Engineering' ;
```

Question 5 : Describe the concept of SQL Normalization and its primary goals. Briefly explain the first three normal forms (1NF, 2NF, 3NF).

Answer: SQL Normalization is a systematic database design technique used to organize tables and columns in a relational database management system (RDBMS. It involves breaking down a large, disorganized table into smaller, logically connected tables to ensure data is stored securely and logically.

Primary Goals of Normalization is to:

- **Eliminate data redundancy**: Prevent storing duplicate data across multiple rows or tables to reduce storage waste.
- **Maintain data integrity**: Establish explicit relationships between fields so updates propagate accurately throughout the database.
- **Prevent data anomalies**: Stop accidental inconsistencies from occurring during daily database operations.
    - *Insertion Anomaly*: Inability to add new records because part of the dependent data does not exist yet.

- *Update Anomaly*: Inconsistent data states created when updating a value in one row but missing it in duplicate rows.
- *Deletion Anomaly*: Unintentionally deleting critical data points when removing an unrelated record from a row.
-

1. First Normal Form (1NF): Atomicity

- A table is in 1NF if every table cell contains exactly one single, indivisible value (atomic values), and each record is completely unique. [1, 2]
- Action: Eliminate multi-valued fields, arrays, comma-separated strings, or repeating groups within a single column. [1, 2]

Example: If an employee table has a single cell containing "555-1234, 555-5678" for phone numbers, it violates 1NF. It must be split so that each individual number occupies its own row or distinct record.

2. Second Normal Form (2NF): Full Functional Dependency

- The table must be in 1NF, and every non-prime column must fully depend on the entire primary key.
- Action: Eliminate partial functional dependencies. This issue happens when a table uses a composite primary key (a primary key made of two or more columns), and a non-key column depends on only *one* part of that composite key instead of the whole combination.
- **Example**: Consider a composite key table with StudentID and CourseID. If you include a CourseName column, it depends solely on CourseID (ignoring StudentID). To achieve 2NF, you must extract CourseID and CourseName into their own separate Courses table.

3. Third Normal Form (3NF): No Transitive Dependency

- The table must be in 2NF, and no non-prime column should depend on any other non-prime column. All fields must strictly depend *only* on the primary key.
- Action: Eliminate transitive functional dependencies. If column A determines column B, and column B determines column C, then column A transitively determines column C.
- **Example**: In an Employees table with EmployeeID (Primary Key), DepartmentID, and DepartmentName, the DepartmentName column actually depends on DepartmentID (a

non-key column) rather than EmployeeID. To satisfy 3NF, move the department details into a dedicated Departments lookup table

---